Weeks 5-6: Deep Analysis of Padding Oracle Attack

1. Context, Risks, and Security Goals

1.1. Context

The Padding Oracle Attack is a classic side-channel attack targeting the implementation of block ciphers. As outlined in the project methodology, this experiment focuses on **AES** operating in **CBC (Cipher Block Chaining)** mode using **PKCS#7** padding. This configuration is historically common in web applications, legacy TLS implementations, and API token encryption.

The core vulnerability lies not in the AES algorithm itself, but in the system's error handling. When a server acts as an "Oracle"—revealing whether a ciphertext has valid padding or not via distinct error messages or timing differences—it leaks critical information about the plaintext.

1.2. Risks Assessment

Based on the deployment weakness analysis, the risks are classified as follows:

- **Confidentiality Loss:** An attacker can decrypt the entire ciphertext (e.g., session cookies, sensitive JSON payloads) without possessing the secret key or the IV.
- **Arbitrary Encryption (Forgery):** In certain variations, the attacker can forge valid ciphertexts, potentially leading to privilege escalation (e.g., changing role from `user` to `admin`).
- **Practicality:** This attack is highly practical in web environments (TLS, APIs) where error verbosity is often enabled for debugging.

1.3. Security Goals

To defend against this vector, the system must achieve:

- **Ciphertext Integrity:** Ensuring that any modification to the ciphertext is detected before decryption attempts (typically via HMAC or AEAD).
- **Indistinguishability:** The system's response (both content and timing) must be identical regardless of whether decryption fails due to invalid padding or other data errors.

2. Solution Architecture & Attack Logic

To satisfy the experimental requirements, the following architecture is proposed:

2.1. The Vulnerable Server (The Oracle)

- **Function:** A REST API endpoint accepting encrypted tokens (hex or base64).
- **Crypto Configuration:** AES-128-CBC with PKCS#7 padding.
- **The Flaw:** The server returns a **500 Internal Server Error** (or specific "Padding Exception") when padding is invalid, but returns a **200 OK** or a generic logic error when padding is valid (even if the decrypted is garbage).

Here is an example of the main function in server using Python:

```python
def func():
    ct = bytes.fromhex(input("Send ciphertext (hex): "))
    try:
        if len(ct) < AES.block_size * 2:
            print(b"Length of ciphertext not valid for CBC mode.")
            return

        pt = cbc_decrypt(ct)
        if pt.decode(errors='ignore') == FLAG:
            print(f"Correct! Here is your flag: {FLAG}")
        else:
            print("Nope, try again!")
    except ValueError:
        print("FAIL: Bad padding.")
    except Exception:
        print(f"FAIL: Invalid input.")
```

2.2. The Attacker (The Client)

- **Logic:** The attack exploits the CBC decryption formula:

$$P_i = D_K(C_i) \oplus C_{j-1}$$

- **Procedure:**

1. The attacker captures a valid ciphertext, isolates a target block C_i and the preceding block C_{i-1} (which acts as the IV for C_i).
2. **Bit-Flipping:** The attacker modifies the last byte of C_{i-1} and sends it to the server.
3. **Oracle Feedback:** If the server returns a "Valid Padding" response, the attacker knows that $D_K(C_i) \oplus C'_{j-1}$ results in a valid padding byte (e.g., 0x01).
4. **Recovery:** The intermediate state and plaintext byte are calculated mathematically. This is repeated for every byte in the block.

Here is the main attack script:

```
def single_block_attack(iv, ciphertext, oracle):
    after_decrypt = b""
    global COUNT
    for i in reversed(range(16)):
        padding = bytes([16 - i] * (16 - i))
        for ch in range(256):
            COUNT += 1
            _iv = bytes(i) + xor(padding, bytes([ch]) + after_decrypt)

            if oracle(_iv, ciphertext):
                after_decrypt = bytes([ch]) + after_decrypt
                break

    return xor(iv, after_decrypt)


def full_attack(iv, ciphertext, oracle):
    plaintext = b""
    for i in range(0, len(ciphertext), BLOCK_SIZE):
        block = single_block_attack(iv, ciphertext[i : i + BLOCK_SIZE], oracle)
        plaintext += block
        iv = ciphertext[i : i + BLOCK_SIZE]
    return plaintext
```

3. Implementation and Evaluation

3.1. Experimental Environment Setup

To accurately measure the severity and resource cost of the attack, we established a controlled environment using **Docker** to simulate a vulnerable production service. The setup ensures distinguishable behavior between "Integrity Errors" and "Padding Errors," which is the prerequisite for the Oracle to exist.

Component	Role	Technical Configuration
Target Server	Vulnerable Oracle	**Ubuntu Docker** container running a service with **AES-CBC-256** and **PKCS#7 Padding**. Configured to return distinguishable error messages/codes.
Attacker Client	Exploit Tooling	Python script utilizing **pwntools** for socket interaction and binary manipulation.
Monitoring	Metrics Collection	Wall-clock timer, request counters, and latency trackers.

Prerequisite: The attacker is assumed to possess a valid **Ciphertext** and the corresponding **Initialization Vector (IV)** captured from a previous session.

3.2. Measured Results

We executed the attack against the local Docker environment to measure the baseline cost of exploitation. The results below demonstrate the low computational cost required to compromise data confidentiality.

Parameter	**Measured Value**	**Notes**
Target Data	5 Blocks (1 IV + 4 Data)	Total of 64 bytes of encrypted data recovered.
Total Oracle Requests	**~ 8,021 requests**	Average of $\approx 128 \times N$ requests.
Wall Time	**~ 38.3424 seconds**	Measured on Localhost (low latency). Real-world WAN attacks would be slower but equally effective.
Success Rate	**100%**	The plaintext was fully recovered with zero errors.

3.4. Patching and Retesting

Following the successful exploitation, we applied two distinct defense strategies and re-ran the attack script to verify their effectiveness.

- **Patch A: Encrypt-then-MAC (HMAC-SHA256)**
    - **Implementation:** We appended an HMAC-SHA256 signature to the ciphertext. The server was reconfigured to verify the MAC *before* attempting decryption.
    - **Result:** The attack failed immediately. The server returned MAC/Integrity Error for every manipulated block, preventing the padding check from ever occurring. **The Oracle was eliminated.**
- **Patch B: Migration to AEAD (AES-GCM)**
    - **Implementation:** The cipher mode was switched from AES-CBC to **AES-GCM**.
    - **Result: Attack failed.** Any modification to the ciphertext invalidated the GCM Tag, causing the decryption to abort before any padding logic could be triggered.

4. Mitigation and Recommendations

Based on the experimental results, the following recommendations are proposed to permanently secure the system against Padding Oracle attacks:

4.1. Prioritize AEAD Modes

- **Recommendation:** Deprecate AES-CBC entirely in favor of **AEAD (Authenticated Encryption with Associated Data)** modes.
- **Standard:** Use **AES-GCM** or **ChaCha20-Poly1305**. These modes enforce integrity by default, making padding oracle attacks mathematically impossible.

4.2. Secure Legacy Implementations

- **Recommendation:** If AES-CBC cannot be replaced due to legacy constraints, strictly implement the **Encrypt-then-MAC** pattern.
- **Requirement:** The MAC must be verified *before* any decryption occurs. If the MAC is invalid, drop the packet immediately without parsing the ciphertext.

4.3. Uniform Error Handling

- **Recommendation:** The server must return a **single, generic error message** for all decryption and authentication failures (e.g., "Invalid Request").
- **Timing:** Ensure constant-time responses to prevent **Timing Attacks** (where an attacker distinguishes errors based on how long the server takes to respond).

4.4. Operational Security

- **Monitoring:** Implement logging to detect high volumes of decryption failures from a single IP, which is a signature of this attack (e.g. >100 errors/minute).
- **Rate-Limiting:** Apply strict rate limits on decryption endpoints to make the 8,000+ requests required for this attack computationally infeasible in a production environment.

5. References

- Wikipedia - Padding Oracle Attack
(https://en.wikipedia.org/wiki/Padding_oracle_attack)

- Cryptopals: Exploiting CBC Padding Oracles By Eli Sohl, 17 February 2021
(https://www.nccgroup.com/research-blog/cryptopals-exploiting-cbc-padding-oracles)

- Juliano Rizzo, Thai Duong, “Practical Padding Oracle Attacks”
(https://www.usenix.org/legacy/event/woot10/tech/full_papers/Rizzo.pdf)